

INTERNET OF THINGS (IoT)

Unit IV — Comprehensive Study Notes

Cloud and Data Management in IoT

TY BSC IT | Exam Preparation Guide

Cognitive Level: K1, K2, K3, K4, K5, K6 | Hours: 12 | PO: 1, 2, 3, 4

- ✓ Cloud Platforms for IoT — ThingSpeak, AWS IoT, Google Cloud IoT
 - ✓ Real-time Data Collection and Analytics
 - ✓ Data Storage and Visualization
 - ✓ Edge and Fog Computing Concepts
 - ✓ Security and Privacy in IoT

1. Cloud Computing Fundamentals for IoT

Cloud Computing — Definition

Cloud computing is the on-demand delivery of IT resources — including computing power, storage, databases, networking, analytics, and software — over the Internet ('the cloud') with pay-as-you-go pricing. Instead of owning and maintaining physical data centers and servers, organisations can access technology services from a cloud provider and pay only for what they use.

1.1 Why Cloud Computing is Essential for IoT

IoT systems generate enormous volumes of data continuously. Local processing (on microcontrollers) is limited by hardware constraints. The cloud provides the unlimited scalability, storage, and analytical power that IoT systems require.

IoT Challenge	How Cloud Solves It
Billions of devices generating data simultaneously	Cloud provides auto-scaling infrastructure — adds capacity automatically as load increases
Limited storage on edge devices	Cloud offers virtually unlimited storage (S3, Azure Blob, GCS Buckets) at low cost
Need for advanced analytics and ML	Cloud provides managed ML services (AWS SageMaker, Google Vertex AI, Azure ML)
Global device management	Cloud IoT platforms offer device registry, OTA updates, remote management at scale
High availability requirement	Cloud providers offer 99.99%+ SLA uptime with redundant data centres worldwide
Collaboration across organisations	Cloud enables multiple stakeholders to access shared IoT data securely
Cost of on-premise infrastructure	Cloud eliminates capital expenditure — pay-as-you-go operational expenditure model
Security expertise requirement	Cloud providers employ dedicated security teams and provide managed security services

1.2 Cloud Service Models for IoT

Service Model	Full Name	What is Managed by Cloud	What IoT Developer Controls	IoT Examples
---------------	-----------	--------------------------	-----------------------------	--------------

IaaS	Infrastructure as a Service	Physical servers, networking, virtualisation, storage hardware	OS, runtime, middleware, data, applications	AWS EC2 running custom IoT broker, Azure VM hosting Grafana dashboard
PaaS	Platform as a Service	IaaS + OS + runtime + middleware + databases	Applications and data only	AWS IoT Core, Azure IoT Hub, Google Cloud IoT Core — managed IoT platforms
SaaS	Software as a Service	Everything including the application	Configuration and data input only	ThingSpeak, Blynk, Ubidots, Salesforce IoT Cloud
FaaS	Functions as a Service (Serverless)	IaaS + PaaS + application scaling	Code functions only — triggered by events	AWS Lambda processing sensor alert, Azure Functions on IoT Hub message

1.3 Cloud Deployment Models

Deployment Model	Description, Pros, Cons & IoT Use Cases
Public Cloud	Infrastructure owned and operated by a third-party provider (AWS, Azure, GCP) and shared among multiple organisations. Pros: No upfront cost, infinite scalability, global availability. Cons: Data leaves organisation premises (compliance concerns), requires internet connectivity, shared resources. IoT Use: Most common for consumer IoT, smart home, wearables.
Private Cloud	Cloud infrastructure operated exclusively for a single organisation — on-premise or hosted. Pros: Full control, maximum security, compliance (healthcare, defence, banking), no data sovereignty issues. Cons: High upfront capital cost, requires IT staff to manage. IoT Use: Industrial IoT with sensitive data (manufacturing, utilities, healthcare).
Hybrid Cloud	Combination of public and private cloud — data and applications move between them. Pros: Balance of control and scalability. Critical/sensitive data on private; scalable compute on public. IoT Use: Smart grid (metering data public cloud, control systems private), connected vehicles (telemetry public, safety control private).
Multi-Cloud	Using services from multiple cloud providers simultaneously. Pros: Avoid vendor lock-in, use best-of-breed services from each provider, geographic redundancy. Cons: Complexity, multiple skill

	sets needed. IoT Use: Large enterprises using AWS IoT + Azure ML + Google BigQuery.
--	---

2. IoT Cloud Platforms — ThingSpeak, AWS IoT, Google Cloud IoT

IoT Cloud Platforms are specialised cloud services designed specifically to handle the unique requirements of IoT: device connectivity, message ingestion, device management, data storage, and analytics at scale. The three most important platforms for your syllabus are ThingSpeak, AWS IoT, and Google Cloud IoT.

2.1 ThingSpeak

ThingSpeak — Definition

ThingSpeak is an open-source IoT analytics platform service developed by MathWorks (makers of MATLAB). It allows users to aggregate, visualise, and analyse live data streams in the cloud. ThingSpeak provides instant data visualisation and built-in MATLAB analytics capabilities. It is the most beginner-friendly IoT cloud platform and ideal for academic projects.

2.1.1 ThingSpeak Architecture & Key Concepts

ThingSpeak Concept	Detailed Explanation
Channel	The fundamental unit of ThingSpeak. A channel stores data from one IoT device or data source. Each channel has up to 8 data fields (Field 1 – Field 8), plus metadata (channel name, description, location). Think of it as a data table for one device.
Field	Each channel has 8 fields to store different sensor readings. Field 1 = Temperature, Field 2 = Humidity, Field 3 = Pressure, etc. Each field holds one numeric or string value per update.
Write API Key	Secret key used by IoT device to SEND (write) data to the channel. Must be kept secret — anyone with this key can write data.
Read API Key	Key used to READ channel data. Can be shared publicly for public channels or kept private for restricted access.
Channel Feed	The time-series record of all data sent to a channel. Accessible via REST API as JSON, CSV, or XML.
Update Rate	Free plan: minimum 15 seconds between updates (throttled). Paid plans allow higher frequencies. Arduino must add delay(15000) between ThingSpeak sends.
MATLAB Analysis	Built-in MATLAB code execution environment triggered by data updates, timers, or on-demand. Can perform statistical analysis, signal processing, and generate computed outputs.

React	ThingSpeak condition-action engine. If Field 1 > 30, send HTTP request (trigger IFTTT, send email, change LED colour). Simple rule-based automation.
TalkBack	Command queue for devices. IoT device polls the TalkBack queue and executes queued commands (e.g., 'TURN_FAN_ON'). Enables bidirectional communication.
TimeControl	Schedule MATLAB code execution at fixed intervals — like a cron job for IoT analytics.

2.1.2 Sending Data to ThingSpeak — Multiple Methods

Method 1 — HTTP GET Request (simplest)

```
// ThingSpeak update URL format:
// GET https://api.thingspeak.com/update?api_key=<KEY>&field1=<VALUE>

#include <WiFi.h>
#include <HTTPClient.h>

String WRITE_API_KEY = "XXXXXXXXXXXXXXXXX"; // Your ThingSpeak Write
API Key

void sendToThingSpeak(float temp, float hum, float soil) {
  String url = "http://api.thingspeak.com/update";
  url += "?api_key=" + WRITE_API_KEY;
  url += "&field1=" + String(temp, 2); // 2 decimal places
  url += "&field2=" + String(hum, 2);
  url += "&field3=" + String(soil);

  HTTPClient http;
  http.begin(url);
  int code = http.GET(); // Returns entry_id if success
  Serial.println("HTTP code: " + String(code)); // 200 = success
  http.end();
  delay(16000); // ThingSpeak free plan: 15s minimum between updates
}
```

Method 2 — MQTT with ThingSpeak

```
// ThingSpeak MQTT Broker: mqtt3.thingspeak.com Port: 1883 (8883 TLS)
// Publish topic format: channels/<channelID>/publish
// Subscribe topic: channels/<channelID>/subscribe/fields/field1

#include <PubSubClient.h>
```

```
const char* mqttServer      = "mqtt3.thingspeak.com";
const char* mqttUsername    = "<ThingSpeak_Username>";
const char* mqttPassword    = "<MQTT_API_Key>"; // Different from Write
API Key
const char* clientID        = "<ThingSpeak_ClientID>";
long channelID              = 1234567;

// Publish payload format: field1=25.4&field2=60.2
String payload = "field1=" + String(temperature) + "&field2=" +
String(humidity);
mqtt.publish("channels/" + String(channelID) + "/publish",
payload.c_str());
```

2.1.3 ThingSpeak — Reading Data via REST API

```
// GET last entry: GET
/channels/<ID>/feeds/json?api_key=<READ_KEY>
// GET last N:      GET
/channels/<ID>/feeds/json?api_key=<KEY>&results=10
// GET field only:  GET /channels/<ID>/fields/1/last.json?api_key=<KEY>
// Date range:      GET /channels/<ID>/feeds.json?start=2024-01-
01&end=2024-01-31

// Example JSON response for last entry:
{
  "created_at": "2024-10-15T14:30:00Z",
  "entry_id": 1547,
  "field1": "28.50", // Temperature
  "field2": "65.20", // Humidity
  "field3": "42"     // Soil Moisture
}
```

2.1.4 ThingSpeak — Key Features Summary

Feature	Free Plan	Student/Academic Plan	Commercial
Channels	4	Unlimited	Unlimited
Update interval	15 seconds	1 second	1 second
Storage	3M messages	Unlimited	Unlimited
MATLAB analysis	60 min/day	Unlimited	Unlimited
HTTP API	Yes	Yes	Yes
MQTT	Yes	Yes	Yes
Data Export	CSV, JSON, XML	CSV, JSON, XML	All formats

Cost	Free	Free (education)	~\$10/month
------	------	------------------	-------------

2.2 AWS IoT Core

AWS IoT Core — Definition

AWS IoT Core is Amazon's fully managed cloud service that enables billions of IoT devices to securely connect and interact with AWS cloud services and each other. It supports MQTT, HTTPS, and WebSocket protocols. AWS IoT Core provides a scalable, serverless message broker, device registry, device shadows, and a powerful rules engine — all without managing servers.

2.2.1 AWS IoT Core — Architecture Components

AWS IoT Core — Core Components

- ▶ IoT Core Message Broker: Fully managed MQTT broker. Supports MQTT 3.1.1 and MQTT 5.0. Handles millions of simultaneous connections. Connects over port 8883 (TLS) or 443 (WebSocket/TLS). Scales automatically — no configuration needed.
- ▶ Device Registry: Central database of all registered IoT devices (called 'things'). Stores device metadata (name, type, attributes) and manages device certificates. Every device gets a unique Amazon Resource Name (ARN).
- ▶ Device Shadow (Thing Shadow): A persistent JSON document stored in the cloud that represents the DESIRED and REPORTED state of a device. Even when a device is offline, other services can read/update the shadow. When the device reconnects, it syncs its state with the shadow.
- ▶ Rules Engine: SQL-like query language that filters incoming MQTT messages and triggers actions. Example: `SELECT temperature FROM sensors/+/data WHERE temperature > 30 → store in DynamoDB AND send SNS alert.`
- ▶ Thing Types & Groups: Organise devices into types (SmartThermostat, SecurityCamera) and groups for bulk management, policy application, and OTA updates.
- ▶ IoT Policies: Fine-grained AWS IAM-like permissions defining what MQTT topics a device can publish to or subscribe from. Critical for zero-trust security.
- ▶ Greengrass (Edge): AWS Greengrass extends AWS IoT Core to edge devices (gateways) — run Lambda functions locally, sync device shadows offline, use ML inference at edge.
- ▶ Device Defender: Continuously audits IoT device configurations and monitors device behaviour for anomalies — detects compromised devices.
- ▶ Fleet Indexing: Index and search device attributes, shadow data, and connectivity status across millions of devices. Query: `SELECT * WHERE thingName LIKE 'Sensor_*' AND shadow.temp > 40`
- ▶ OTA Updates (Jobs): AWS IoT Jobs service manages Over-The-Air firmware updates to devices at scale — tracks status (queued, in-progress, succeeded, failed) across entire fleets.

2.2.2 AWS IoT Device Shadow — Deep Dive

Device Shadow — Definition

A Device Shadow is a JSON document stored in AWS IoT Core that acts as a virtual representation (twin) of a physical IoT device in the cloud. It stores the last known REPORTED state of the device and the DESIRED state that applications want to achieve. The shadow persists even when the device is offline — enabling cloud-to-device communication reliably.

Device Shadow JSON Structure:

```
// Device Shadow Document Structure
{
  "state": {
    "desired": {           // What the app WANTS the device state to be
      "led": "ON",
      "fan_speed": 75,
      "temp_threshold": 28
    },
    "reported": {         // What the device ACTUALLY IS currently
      "led": "OFF",       // Device hasn't applied the desired change yet
      "fan_speed": 50,    // Mismatch = device needs to update
      "temperature": 24.5,
      "uptime_hours": 127
    },
    "delta": {            // AWS auto-calculates: desired != reported
      "led": "ON",       // Delta = what device needs to change
      "fan_speed": 75
    }
  },
  "metadata": { ... },   // Timestamps for each field update
  "version": 42,         // Increments on every shadow update
  "timestamp": 1699000000 // Unix timestamp of last update
}
```

Device Shadow MQTT Topics:

MQTT Topic	Purpose
\$aws/things/{name}/shadow/update	Device/App publishes to UPDATE the shadow (set desired or reported state)
\$aws/things/{name}/shadow/update/accepted	Broker publishes here if update was accepted and processed successfully
\$aws/things/{name}/shadow/update/rejected	Broker publishes here if update failed (version conflict, invalid JSON)

\$aws/things/{name}/shadow/update/delta	Broker publishes here when desired != reported — device subscribes to get pending changes
\$aws/things/{name}/shadow/get	Device publishes empty message here to REQUEST its current shadow document
\$aws/things/{name}/shadow/get/accepted	Response to GET — shadow document published here
\$aws/things/{name}/shadow/delete	Delete the shadow document entirely

2.2.3 AWS IoT Rules Engine — SQL for IoT Data

The Rules Engine is one of AWS IoT Core's most powerful features. It processes every incoming MQTT message using SQL queries and can trigger up to 10 actions per rule:

```
-- Rule 1: Store ALL temperature data in DynamoDB time-series table
SELECT deviceId, temperature, humidity, timestamp()
FROM 'sensors/+/telemetry'
WHERE temperature IS NOT NULL

-- Rule 2: Trigger Lambda when temperature is dangerously HIGH
SELECT *, topic() AS sourceTopic, timestamp() AS eventTime
FROM 'factory/+/sensors'
WHERE temperature > 80 AND machineStatus = 'RUNNING'

-- Rule 3: Send to Kinesis stream for real-time analytics pipeline
SELECT * FROM 'devices/+/readings' WHERE reported.anomaly = true

-- Rule 4: Forward to S3 for long-term storage (data lake)
SELECT encode(*, 'base64') FROM 'cameras/+/frames'
```

AWS IoT Rules Engine — Available Actions:

Action	What it Does
DynamoDB	Store IoT data as items in a DynamoDB table — perfect for time-series sensor data with low-latency queries
DynamoDBv2	Store entire MQTT message as a DynamoDB item with custom key mapping
S3	Store raw data (JSON, binary, images) in S3 bucket — data lake for long-term retention and batch analytics
Lambda	Invoke a serverless Lambda function with the message — custom business logic, ML inference, HTTP calls
SNS	Publish to SNS topic — sends notifications via email, SMS, mobile push, or HTTP webhook

SQS	Send to SQS queue — decouple IoT data ingestion from processing (buffer for downstream services)
IoT Analytics	Send to AWS IoT Analytics pipeline — automated data cleansing, enrichment, and analytics
Kinesis Data Streams	Stream IoT data to Kinesis for real-time analytics (Kinesis Analytics) or data lake (Kinesis Firehose → S3)
Elasticsearch / OpenSearch	Index IoT data for real-time search and analytics via Kibana/OpenSearch Dashboards
Timestream	Store data in AWS Timestream purpose-built time-series database — optimised for IoT sensor data

2.2.4 AWS IoT Core — Complete Connection Example (Python)

```
# Install: pip install awsiotsdk
from awsiot import mqtt_connection_builder
import json, time

# AWS IoT endpoint (found in AWS Console → IoT Core → Settings)
ENDPOINT = "a1b2c3d4e5f6g7.iot.us-east-1.amazonaws.com"
CLIENT_ID = "MyRaspberryPi"
TOPIC = "home/sensors/temperature"

# X.509 certificate paths (downloaded from AWS IoT Console)
mqtt_connection = mqtt_connection_builder.mtls_from_path(
    endpoint = ENDPOINT,
    cert_filepath = "device-certificate.pem.crt",
    pri_key_filepath = "private.pem.key",
    ca_filepath = "AmazonRootCA1.pem",
    client_id = CLIENT_ID,
    clean_session = False,
)

connect_future = mqtt_connection.connect()
connect_future.result() # Block until connected
print("Connected to AWS IoT Core!")

# Publish sensor data every 10 seconds
while True:
    payload = json.dumps({
        "deviceId" : CLIENT_ID,
        "temperature" : 24.5,
        "humidity" : 65.2,
        "timestamp" : int(time.time())
    })
    mqtt_connection.publish(topic=TOPIC, payload=payload, qos=1)
```

```
print(f"Published: {payload}")
time.sleep(10)
```

2.3 Google Cloud IoT

Google Cloud IoT Core — Definition

Google Cloud IoT Core was a fully managed service for securely connecting and managing IoT devices at scale. While Google deprecated IoT Core in August 2023, Google's broader IoT strategy now revolves around integrating devices with Cloud Pub/Sub, BigQuery, Vertex AI, and Firebase. The concepts remain highly relevant for understanding enterprise IoT architectures.

2.3.1 Google Cloud IoT Architecture

Google Cloud IoT Data Pipeline

- ▶ **Device Layer:** IoT devices connect to Cloud IoT Core using MQTT (port 8883) or HTTPS. Each device is authenticated using RSA or Elliptic Curve cryptography (JWT tokens signed with device's private key — unique approach vs AWS X.509 certificates).
- ▶ **Cloud Pub/Sub:** Google's fully managed real-time messaging service. IoT Core automatically forwarded all device telemetry to a Pub/Sub topic. Cloud Pub/Sub can handle millions of messages per second with guaranteed delivery and replay capability.
- ▶ **Cloud Dataflow:** Managed Apache Beam service for real-time stream processing. Processes IoT data from Pub/Sub — applies transformations, windowing (tumbling, sliding, session windows), anomaly detection, and routes to storage.
- ▶ **BigQuery:** Google's serverless, petabyte-scale data warehouse. Stores all historical IoT data. Allows SQL queries over billions of records in seconds. Best for analytics, reporting, and ML training datasets.
- ▶ **Cloud AI Platform / Vertex AI:** Train and deploy ML models on IoT data. AutoML for time-series prediction, anomaly detection, predictive maintenance models deployed back to edge via Edge TPU.
- ▶ **Firebase Realtime Database / Firestore:** NoSQL cloud database for real-time IoT dashboard applications. Changes propagate to all connected clients in milliseconds — ideal for live dashboards showing current sensor values.
- ▶ **Cloud Functions:** Serverless event-driven functions triggered by Pub/Sub messages — equivalent to AWS Lambda. Process IoT events, send notifications, update databases, call external APIs.
- ▶ **Looker Studio (Data Studio):** Business intelligence tool for creating IoT dashboards and reports from BigQuery data. Drag-and-drop dashboard creation with real-time BigQuery connectivity.

2.3.2 Google Cloud IoT — Key Technologies

Google Cloud Service	Role in IoT Pipeline
----------------------	----------------------

Cloud Pub/Sub	Message ingestion backbone. Publisher (IoT device) → Pub/Sub topic → multiple subscribers (Dataflow, Cloud Functions, BigQuery). Durable — stores messages for up to 7 days.
BigQuery	IoT data warehouse. Ingest millions of rows/second via streaming inserts. Run SQL analytics: <code>SELECT AVG(temperature) FROM iot.sensors WHERE DATE(timestamp) = '2024-10-15' AND device_id = 'greenhouse_01'</code>
Cloud Dataflow	Real-time ETL for IoT. Reads from Pub/Sub, applies windowed aggregations, detects anomalies, enriches data with reference tables, writes to BigQuery / Bigtable / Spanner.
Firebase	Real-time database for IoT apps. Values update across all clients in <100ms. IoT device writes to Firebase → mobile app dashboard updates instantly. Perfect for consumer IoT apps.
Cloud Bigtable	HBase-compatible NoSQL database optimised for time-series data. Stores trillions of sensor readings at very low latency. Ideal for smart meters, financial IoT, genomics.
Vertex AI	Managed ML platform. Train anomaly detection models on sensor time-series. Deploy as REST API endpoint for real-time inference. AutoML for non-ML-experts.
Cloud Spanner	Globally distributed relational SQL database. For IoT applications requiring global consistency (supply chain, banking IoT) across multiple regions.
Cloud Storage (GCS)	Object storage for raw IoT data archival (images, video, large files). Very cheap cold storage tier (Nearline/Coldline) for regulatory data retention.

2.3.3 AWS IoT vs Google Cloud IoT vs Azure IoT Hub — Platform Comparison

Feature	AWS IoT Core	Google Cloud IoT	Azure IoT Hub
Status (2024)	Active — market leader	IoT Core deprecated; use Pub/Sub	Active — strong enterprise
Device Protocol	MQTT, HTTPS, WebSocket	MQTT, HTTPS	MQTT, AMQP, HTTPS
Device Auth	X.509 certificates	JWT + RSA/ECC keys	SAS tokens or X.509 certs
Edge Computing	AWS Greengrass	Google Distributed Cloud Edge	Azure IoT Edge
Rules Engine	SQL-based, 10+ action targets	Pub/Sub + Cloud Functions	Message routing + Event Grid
Device Twin	Device Shadow (JSON)	Device config + state (deprecated)	Device Twin (desired/reported JSON)
OTA Updates	AWS IoT Jobs	Via Cloud Functions	Device Update for IoT Hub

Analytics	IoT Analytics, Timestream	BigQuery, Dataflow	Azure Stream Analytics, ADX
ML/AI Integration	SageMaker	Vertex AI, AutoML	Azure ML, Cognitive Services
Database	DynamoDB, RDS, Timestream	BigQuery, Bigtable, Firestore	Cosmos DB, Azure SQL, ADX
Free Tier	250K msgs/month	250M Pub/Sub msgs/month	8,000 msgs/day IoT Hub F1
Best For	Broad AWS ecosystem users	ML-heavy, analytics-first IoT	Enterprise, Microsoft stack

3. Real-time Data Collection and Analytics

Real-time IoT Analytics — Definition

Real-time IoT analytics is the process of continuously analysing IoT data streams as they are generated — with latency measured in milliseconds to seconds — to derive immediate, actionable insights. Unlike batch analytics (which processes historical data), real-time analytics detects anomalies, triggers alerts, and enables automated responses before problems escalate.

3.1 IoT Data Characteristics — The 5 Vs

V	Characteristic	IoT Context	Challenge
Volume	Massive quantity of data	Billions of sensors generating readings every second. IDC forecasts 79 ZB of IoT data by 2025.	Cannot store everything — must aggregate, filter, summarise. Requires auto-scaling storage.
Velocity	Speed of data generation/processing	Real-time streams: factory sensors at 1000 Hz, smart cameras at 30 fps, smartwatch at 50 Hz heart rate.	Must process data as fast as it arrives — stream processing, not batch. Low-latency pipelines essential.
Variety	Different data types and formats	Structured (sensor numbers), semi-structured (JSON/XML), unstructured (images, audio, video from IoT cameras).	Unified platform must handle all formats. Normalisation and schema management needed.
Veracity	Quality and accuracy of data	Sensors malfunction, drift, produce noise or outliers. Network packet loss causes missing data.	Data cleaning, anomaly detection, duplicate removal, gap filling in time-series.
Value	Business value extracted from data	Raw temperature readings have little value alone. Predictive maintenance models save millions in downtime.	Must apply analytics, ML, and visualisation to extract actionable insights from raw numbers.

3.2 IoT Data Collection Architecture

3.2.1 Data Ingestion Layer

The data ingestion layer is the entry point for IoT data into the analytics pipeline. It must handle high throughput and maintain data order:

Ingestion Service	Description & IoT Use Case
-------------------	----------------------------

AWS Kinesis Data Streams	Real-time data stream. Ingests millions of records per second from IoT devices. Retains data for 1–365 days. Multiple consumers can read the same stream independently. Use: Real-time anomaly detection while simultaneously archiving to S3.
Apache Kafka	Open-source distributed streaming platform. Topics, partitions, consumer groups. Industry standard for high-throughput IoT pipelines. Confluent Cloud = managed Kafka. Use: Enterprise IoT with complex processing requirements.
AWS Kinesis Data Firehose	Managed delivery service. Automatically loads streaming IoT data into S3, Redshift, OpenSearch, or Splunk. Auto-scales. No consumer management needed. Use: Simplest IoT → data lake pipeline.
Azure Event Hubs	Microsoft's managed Kafka-compatible event streaming. Processes millions of events per second. Integrates with Azure Stream Analytics for real-time processing. Use: Industrial IoT on Microsoft stack.
Google Cloud Pub/Sub	Google's fully managed asynchronous messaging. IoT devices publish to topics; multiple services subscribe. Global, with guaranteed delivery. Use: Google Cloud IoT data ingestion and fan-out.
MQTT Broker + Bridge	On-premises MQTT broker (Mosquitto, EMQX) bridges to cloud. Provides local buffering if cloud connectivity is interrupted. Use: Hybrid edge-cloud architectures.

3.2.2 Stream Processing

Stream Processing — Definition

Stream processing is the continuous analysis of data in motion — processing each event or record as it arrives, without waiting to accumulate a batch. For IoT, this enables sub-second detection of anomalies, threshold violations, equipment failures, and security breaches.

Stream Processing Concept	Explanation for IoT
Windowing	Group stream events into time-based batches for aggregation. Tumbling window: non-overlapping fixed intervals (calculate avg temp every 5 minutes). Sliding window: overlapping windows (rolling average over last 10 minutes, updated every 1 minute). Session window: group events by activity periods.
Stateful Processing	Processing that remembers previous events. Example: Count consecutive temperature readings above threshold — requires remembering previous values. Managed by state stores (RocksDB in Flink/Kafka Streams).
Event Time vs Ingestion Time	Event Time: when the sensor actually recorded the measurement. Ingestion Time: when the data arrived at the

	server. For IoT, always prefer Event Time — accounts for network delays and out-of-order delivery.
Watermarks	Mechanism to handle late-arriving events. A watermark declares 'all events up to time T have arrived'. Allows windowed operations to close and produce results despite late data.
CEP (Complex Event Processing)	Detects patterns across multiple events. Example: 'temperature rising by >2°C per minute AND vibration increasing AND pressure dropping' = machine failure imminent. Apache Flink CEP pattern API.
AWS Kinesis Analytics	Managed stream processing using SQL or Java/Python (Apache Flink). Example: <code>SELECT AVG(temperature) FROM sensor_stream WINDOWED BY TUMBLE(60) GROUP BY deviceId</code> . Triggers Lambda on anomaly.

3.2.3 Batch Analytics vs Real-time Analytics — Comparison

Aspect	Batch Analytics (Lambda Architecture)	Real-time / Stream Analytics (Kappa Architecture)
Data processed	Stored historical data (hours/days old)	Events as they arrive (milliseconds old)
Latency	Minutes to hours	Milliseconds to seconds
Tools	Apache Spark, Hadoop MapReduce, BigQuery	Apache Flink, Kafka Streams, Kinesis Analytics
Use case	Monthly reports, trend analysis, ML training	Anomaly detection, alerts, real-time dashboards
Volume handled	Can process unlimited historical data	Limited to current stream rate
Complexity	Simpler (no state management)	More complex (windowing, state, watermarks)
Cost	Pay per job run	Pay for continuous compute
IoT Example	Nightly analysis of factory energy usage	Immediate alert when machine temperature exceeds threshold

4. Data Storage and Visualisation

IoT generates vast quantities of time-stamped sensor data that must be stored efficiently for both real-time access and long-term historical analysis. The choice of storage technology depends on query patterns, data volume, required latency, and retention needs.

4.1 Database Types for IoT Data

Database Type	Examples	Best For IoT	Query Pattern	Limitations
Time-Series DB	InfluxDB, TimescaleDB, AWS Timestream, Victoria Metrics	Sensor readings with timestamps — the IDEAL for IoT telemetry	<code>SELECT mean(temp) FROM sensors WHERE time > now()-1h GROUP BY time(5m)</code>	Not suited for complex relational queries or documents
NoSQL Document DB	MongoDB, AWS DynamoDB, Google Firestore, Couchbase	Flexible device metadata, user profiles, event logs with varying schemas	<code>db.sensors.find({'temperature': {'\$gt': 30}}).limit(100)</code>	Less efficient for pure time-series; range queries expensive
Relational (SQL) DB	PostgreSQL + TimescaleDB, MySQL, AWS RDS	Device registry, user accounts, relationships between entities	<code>SELECT d.name, AVG(s.temp) FROM sensors s JOIN devices d ON...</code>	Poor performance at IoT scale without time-series extension
Wide-Column Store	Apache Cassandra, Google Bigtable, HBase	Massive scale time-series; trillions of rows; write-heavy IoT	Row key = deviceID+timestamp; column = metric name	No ACID transactions; eventual consistency; complex data model
In-Memory DB	Redis, Memcached, AWS ElastiCache	Real-time latest values (device current state cache), session data, rate limiting	<code>GET device:001:temperature</code> → "24.5" (microsecond latency)	Volatile (data lost on restart unless persistence enabled); expensive per GB
Data Lake / Object	AWS S3, Azure Data Lake Gen2, Google GCS	Raw data archival, ML training datasets, long-term retention (years)	Via Athena/BigQuery federated queries (slow but cheap)	Not a database — no real-time queries; analytical use only

4.2 InfluxDB — Time-Series Database for IoT

InfluxDB — Definition

InfluxDB is an open-source, purpose-built time-series database developed by InfluxData. It is designed specifically for storing and querying time-stamped data at high speed. InfluxDB uses a unique data model and query language (InfluxQL or Flux) optimised for time-series operations like downsampling, aggregation, and retention policy management.

InfluxDB Concept	Explanation
Measurement	Equivalent to a table in SQL. Example: 'sensor_readings', 'weather_data', 'machine_metrics'. All data in a measurement shares the same schema.
Tag	Indexed metadata (string key-value pairs) used for filtering. Example: device_id='sensor_01', location='greenhouse', type='temperature'. Tags are indexed = fast filtering.
Field	Actual numeric or string measurement values (NOT indexed). Example: temperature=24.5, humidity=65.2, pressure=1013.25. Fields are not indexed.
Timestamp	Nanosecond-precision Unix timestamp. The most important column in any time-series record. Auto-set if not provided.
Retention Policy	Defines how long data is kept before automatic deletion. Example: raw data = 30 days, hourly averages = 1 year, daily averages = 5 years. Essential for managing storage costs.
Continuous Query	Automatically downsamples high-frequency data. Example: every 5 minutes, calculate and store the average of the last 5 minutes of 1-second readings. Reduces storage while preserving trends.
Line Protocol	Simple text format for writing data: measurement,tag1=val1,tag2=val2 field1=val1,field2=val2 timestamp. Example: sensors,device=esp32_01,room=bedroom temp=24.5,humidity=65 1699000000000000000

InfluxDB Line Protocol & Flux Query Examples:

```
# Writing data (Line Protocol format):
# measurement,tag_key=tag_value field_key=field_value timestamp
sensors,device_id=esp32_01,location=greenhouse
temperature=24.5,humidity=65.2 1699000000
sensors,device_id=esp32_02,location=bedroom
temperature=22.1,humidity=58.7 1699000000

# Flux Query - average temperature per device over last 24 hours:
from(bucket: "iot_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensors")
  |> filter(fn: (r) => r._field == "temperature")
```

```

|> group(columns: ["device_id"])
|> mean() // Calculate mean per device

# Flux Query — detect temperature spikes (> 35°C):
from(bucket: "iot_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensors" and r._field ==
"temperature")
  |> filter(fn: (r) => r._value > 35.0) // Only anomalous readings
  |> yield(name: "temperature_spikes")

```

4.3 Data Visualisation for IoT

IoT Data Visualisation — Definition

IoT data visualisation is the graphical representation of sensor data, device states, and analytics results to enable humans to quickly understand patterns, trends, anomalies, and system status. Effective visualisation transforms raw numeric data into actionable intelligence through dashboards, charts, maps, and alerts.

4.3.1 Grafana — The Standard IoT Dashboard Tool

Grafana

- ▶ What it is: Open-source analytics and visualisation platform. Grafana is the industry standard for IoT dashboards. Connects to virtually any data source and renders beautiful interactive dashboards.
- ▶ Data Sources: InfluxDB, Prometheus, Elasticsearch, MySQL, PostgreSQL, AWS CloudWatch, Azure Monitor, Google Sheets, MQTT via plugins — all connectable to a single dashboard.
- ▶ Panel Types: Time series graphs (sensor data over time), Stat panels (current value with colour threshold), Gauge (speedometer-style), Bar chart, Heatmap (patterns across devices/time), Geomap (device locations), Alert list.
- ▶ Alerting: Define threshold rules (temperature > 35°C for 5 minutes → alert). Notify via email, Slack, PagerDuty, webhook, SMS (via OpsGenie). Alert state tracking and silencing.
- ▶ Dashboard Variables: Dynamic dropdowns to filter by device_id, location, or time range — one dashboard works for all devices.
- ▶ Grafana Cloud: Managed Grafana hosting. Free tier includes 10,000 metrics, 50GB logs, 50GB traces. Connect your InfluxDB or Prometheus endpoint.
- ▶ Annotations: Mark important events on time-series graphs (firmware updates, maintenance windows, incidents) to correlate with data changes.
- ▶ IoT Example: Dashboard showing: Top row — current temperature/humidity/CO2 stat panels. Middle — 24h time-series graph. Bottom — heatmap of temperatures across all rooms. Right panel — alert list for threshold violations.

4.3.2 ThingSpeak Visualisation

ThingSpeak provides built-in charts directly within the channel view:

- **Channel Charts:** Auto-generated line chart for each field. Configurable time range, chart type (line, column, bar, spline), colour, and labels.
- **MATLAB Visualisation:** Custom charts using MATLAB code — scatter plots, histograms, correlation matrices, 3D plots of multi-sensor data.
- **Gauge Widget:** Display current sensor value as a speedometer gauge on the public channel view.
- **Video Widget:** Embed live video streams from IP cameras alongside sensor data.
- **Public Channel:** Share ThingSpeak channel publicly — anyone can view the real-time data and charts via a URL (no login required).

4.3.3 Other IoT Visualisation Platforms

Platform	Type	Best For	Key Feature
Grafana	Open-source dashboard	Professional IoT dashboards, industrial monitoring	Universal data source connector, alerting
ThingSpeak	Cloud IoT platform	Academic projects, beginners, MATLAB users	Built-in MATLAB analytics, free tier
Kibana	ELK Stack dashboard	Log analytics, IoT anomaly investigation	Full-text search, drill-down analytics
Node-RED	Low-code flow editor	Rapid IoT prototyping, dashboard on RPi	Visual flow programming + built-in UI
Blynk	IoT app builder	Mobile app dashboard for maker/hobbyist IoT	Drag-and-drop mobile UI builder
Ubidots	IoT analytics platform	Industrial IoT, predictive maintenance dashboards	Device management + dashboard in one
Tableau	Business intelligence	Executive IoT reporting, supply chain visibility	Advanced analytics and storytelling
Power BI	Microsoft BI tool	Enterprise IoT KPI dashboards for management	Azure IoT Hub direct connector
Looker Studio	Google BI tool	Google Cloud IoT, BigQuery-backed dashboards	Free, directly queries BigQuery

5. Edge and Fog Computing

5.1 Edge Computing

Edge Computing — Definition

Edge computing is a distributed computing paradigm that brings computation and data storage physically closer to the sources of data — at the 'edge' of the network, near IoT devices — rather than relying on a centralised cloud data centre. Processing happens on the device itself (on-device/embedded ML) or on a nearby local server (gateway edge computing).

5.1.1 Why Edge Computing for IoT — Critical Use Cases

Limitation of Cloud-Only IoT	How Edge Computing Solves It
High latency (50–200ms round-trip to cloud) — unacceptable for autonomous vehicles or surgical robots	Edge processes locally in <5ms — latency reduced by 10–40× for time-critical decisions
Bandwidth cost — sending 30fps HD video from 1000 cameras to cloud is financially and physically impractical	Edge compresses/analyses video locally, only sends metadata or alerts to cloud — 99% bandwidth reduction
Cloud dependency — factory shuts down if internet fails	Edge devices operate fully autonomously offline — sync to cloud when connectivity returns
Privacy regulations — GDPR, HIPAA prohibit patient data leaving local hospital network	Sensitive data processed and stored locally — only anonymised aggregates sent to cloud
Real-time control requirements — industrial robots, CNC machines need deterministic microsecond responses	On-device ML inference in <1ms — no cloud round-trip, no jitter, deterministic timing
Cloud compute costs — continuous stream analytics for billions of devices is prohibitively expensive	Filter and aggregate at edge — send only meaningful events to cloud, drastically reducing cloud cost

5.1.2 Levels of Edge Computing

Level	Location	Hardware	Processing Capability	Examples
Level 0 — Device Edge	On the IoT device itself	MCU (ESP32, STM32) or SoC	Simple rules, TF Lite Micro, anomaly	ESP32 with TF Lite Micro

		with ML accelerator	detection, feature extraction	detecting keyword in audio
Level 1 — Near Edge	On a local gateway (0–10m)	RPi, industrial PC, NVIDIA Jetson Nano/Xavier	Complex ML inference, local MQTT broker, protocol translation	NVIDIA Jetson running object detection on factory camera feed
Level 2 — Far Edge / Micro Data Centre	On-site server (~10–100m)	Ruggedised server rack, AWS Outposts, Azure Stack Edge	Full analytics stack, database, dashboard, ML training on local data	AWS Outposts in a factory — full AWS services running on-premise
Level 3 — Regional Edge / CDN Edge	Telecom data centre (~city)	Cloudflare Workers, AWS Lambda@Edge, telecom MEC	Low-latency compute close to user clusters	5G Multi-access Edge Computing (MEC) for connected vehicles

5.1.3 Edge AI / TinyML

TinyML — Definition

TinyML (Tiny Machine Learning) is the practice of running ML inference models on extremely resource-constrained devices — microcontrollers with as little as 256KB of RAM and no GPU. Frameworks like TensorFlow Lite Micro, Edge Impulse, and ONNX Runtime Mobile enable deploying trained neural networks directly on IoT devices for on-device intelligence.

- **TensorFlow Lite Micro (TFLM):** Google's framework for running TensorFlow models on microcontrollers (Cortex-M series, ESP32, Arduino Nano 33 BLE). Model size: typically 50–500KB.
- **Edge Impulse:** End-to-end platform for collecting IoT sensor data, training ML models (classification, anomaly detection, object detection), and deploying to edge devices via C++ library or Arduino library.
- **ONNX Runtime:** Cross-framework ML inference runtime. Deploy models trained in PyTorch or scikit-learn to edge devices.
- **Hardware Accelerators:** Coral Edge TPU (USB or M.2) — 4 TOPS at 2W. NVIDIA Jetson Nano — 472 GFLOPS. Qualcomm AI Engine in mobile SoCs. Dedicated ML chips dramatically accelerate inference.
- **Use Cases:** Wake word detection (Alexa/Siri without cloud), industrial vibration anomaly detection, image classification on smart cameras, predictive maintenance on CNC machines.

5.2 Fog Computing

Fog Computing — Definition

Fog computing is a distributed computing architecture introduced by Cisco (2014) that extends cloud computing to the edge of the network through an intermediate layer of 'fog nodes' positioned between IoT devices and the cloud. Fog nodes provide compute, storage, and networking services closer to devices than the centralised cloud, but with more processing power than individual edge devices.

5.2.1 Fog Computing Architecture

Fog Computing Three-Layer Architecture

LAYER 3 — CLOUD: Centralised data centres (AWS, Azure, GCP). Global analytics, long-term storage, ML training, business intelligence. Receives aggregated, processed summaries from fog nodes.

LAYER 2 — FOG NODES: Intermediate processing layer — smart gateways, routers with compute, local servers, telecom base stations. Functions: Local analytics, short-term caching, device management, protocol translation, filtering. Distance: metres to kilometres from devices. Latency: 1–10 ms.

LAYER 1 — IoT DEVICES: Sensors, actuators, embedded systems. Generate raw data. Simple local rules only. Extremely resource-constrained. Communicate with fog nodes via Zigbee, BLE, Z-Wave, LoRa, or Ethernet.

DATA FLOW: Device data → Fog node (local processing, immediate response) → aggregated insights → Cloud (long-term analytics, ML training)

5.2.2 Edge vs Fog vs Cloud — Comprehensive Comparison

Property	Edge Computing	Fog Computing	Cloud Computing
Location	On the device or 1-hop gateway	Intermediate nodes between device and cloud	Centralised remote data centres (AWS, GCP)
Latency	< 1–5 ms (on-device) or < 10ms (gateway)	1–50 ms	50–500 ms (internet round-trip)
Bandwidth use	Minimum — processes locally	Reduced — aggregates/filters for cloud	High — raw data sent to cloud
Scale	Limited (device/gateway hardware)	Moderate (distributed fog infrastructure)	Unlimited (cloud auto-scales)
Intelligence	Simple rules + TinyML	Complex analytics, stream processing, ML	Deep ML, petabyte analytics, AI training
Connectivity	Can work fully offline	Works with intermittent cloud connectivity	Requires continuous internet

Cost	Low device cost, no per-request fees	Medium (fog node hardware + networking)	High for large data volumes
Security	Data never leaves premises	Local processing, selective cloud sync	Data sent to third-party provider
Examples	ESP32 TF Lite, NVIDIA Jetson local camera	Smart gateway, 5G MEC, OpenFog nodes	AWS IoT, Google Cloud, Azure IoT Hub

5.2.3 OpenFog Consortium — Fog Computing Standards

OpenFog Consortium: Formed in 2015 by ARM, Cisco, Dell, Intel, Microsoft, and Princeton University to develop open fog computing standards. Now merged with IEEE (IEEE 1934). Defines reference architectures, use cases, and interoperability requirements for fog computing in IoT.

- **SCALE Pillars:** Security, Cognition, Agility, Latency, Efficiency — the five key requirements of fog computing architectures.
- **OpenFog RA (Reference Architecture):** Hierarchical node structure with programmatic management, open interfaces, and RAS (Reliability, Availability, Serviceability) requirements.

6. Security and Privacy in IoT

Security is the greatest challenge facing IoT deployments. The combination of billions of resource-constrained devices, diverse communication protocols, physical accessibility, long deployment lifetimes, and insufficient vendor security practices creates an enormous attack surface. Understanding IoT security is critical for the exam and for real-world IoT development.

6.1 Why IoT Security is Uniquely Challenging

IoT Security Challenge	Why It's Unique Compared to Traditional IT
Resource constraints	Microcontrollers have limited CPU/RAM/energy — cannot run full TLS, certificate management, or complex encryption algorithms designed for powerful computers.
Scale of deployment	Securing 10 servers is manageable. Securing 10 billion IoT devices with individual certificates, patches, and configurations is an unprecedented challenge.
Long deployment lifetime	Industrial sensors may operate for 15–20 years. Firmware vulnerabilities discovered years after deployment cannot be quickly patched without physical access or OTA update infrastructure.
Physical accessibility	IoT devices are physically accessible to attackers (in public spaces, buildings, parking lots). Physical attacks (side-channel, fault injection) are feasible.
Diverse protocols	IoT uses dozens of communication protocols (MQTT, CoAP, Zigbee, LoRa, BLE) — each with its own security model and vulnerabilities.
Default credentials	Many IoT devices ship with default usernames/passwords (admin/admin, root/root). Mirai botnet (2016) compromised 600,000 devices using only default credential scanning.
Lack of security updates	Many IoT manufacturers provide no update mechanism or stop providing security patches after 1–2 years.
Privacy concerns	IoT devices collect highly personal data (health metrics, home occupancy, location, audio/video) — privacy violations have severe consequences.

6.2 IoT Security Threats and Attacks

Attack Type	Description	Real-World Example	Impact
Botnet / DDoS	Compromised IoT devices used to launch distributed denial-of-service	Mirai botnet (2016) used 600K cameras and DVRs to attack Dyn DNS — took	Internet service outages; device hijacking

	attacks against internet services	down Twitter, Netflix, GitHub for hours	
Man-in-the-Middle (MitM)	Attacker intercepts and potentially modifies communication between IoT device and cloud	Intercepting unencrypted HTTP (not HTTPS) sensor data to change temperature thresholds in HVAC	Data manipulation; false commands to actuators
Eavesdropping	Passive interception of unencrypted wireless IoT communications	Capturing unencrypted MQTT messages to learn home occupancy patterns from smart light usage	Privacy violation; intelligence gathering
Replay Attack	Capturing and re-sending valid authenticated IoT messages to repeat commands	Capturing 'unlock door' BLE command and replaying it to re-open a smart lock	Unauthorised physical access
Firmware Attack	Modifying device firmware via physical access, OTA hijacking, or supply chain compromise	2022: IoT smart plugs shipped with pre-installed malware from manufacturer	Complete device takeover
Side-Channel Attack	Extracting cryptographic keys by analysing power consumption or electromagnetic emissions during crypto operations	Extracting AES key from smart card/embedded system via power analysis	Cryptographic key compromise
Physical Tampering	Direct physical manipulation of the device (JTAG debugging, memory readout, sensor spoofing)	Replacing legitimate Zigbee sensor with malicious device broadcasting false readings	False data injection; system manipulation
Cloud/API Attack	Attacking the cloud backend or API rather than the device itself	API key leakage exposing all device data; insecure direct object reference exposing other users' device data	Mass data breach; all devices compromised

6.3 IoT Security Principles and Best Practices

6.3.1 Security by Design — The 7 Pillars

Security Pillar	Principle	IoT Implementation
1. Authentication	Verify the identity of every device and user before granting access	X.509 certificates per device (AWS IoT); JWT tokens; hardware security modules (HSM); TPM chips; SIM-based identity (eSIM)

2. Authorisation	Grant minimum required permissions — principle of least privilege	AWS IoT Policies allowing only specific MQTT topics; role-based access control (RBAC); attribute-based access control
3. Encryption in Transit	Protect data from eavesdropping and MitM attacks during transmission	TLS 1.3 for MQTT (port 8883) and HTTPS; DTLS for CoAP; WPA3 for Wi-Fi; AES-128 for Zigbee/LoRaWAN
4. Encryption at Rest	Protect stored data from physical access and insider threats	Full-disk encryption on IoT gateways (dm-crypt, BitLocker); encrypted database fields; S3 SSE; DynamoDB encryption
5. Integrity	Detect unauthorised modification of data or firmware	Digital signatures on firmware (RSA/ECDSA); HMAC on sensor data; TLS record integrity; Secure Boot
6. Availability	Ensure IoT services remain operational despite attacks	DoS mitigation; rate limiting; redundant cloud regions; watchdog timers; circuit breakers
7. Privacy	Protect personal data per regulations (GDPR, HIPAA, CCPA)	Data minimisation; anonymisation; consent management; data residency controls; right to deletion

6.3.2 Device-Level Security Measures

Security Measure	Description and Implementation
Secure Boot	Cryptographically verify firmware signature before execution. Device refuses to boot if firmware is tampered. ESP32 supports secure boot V2 using RSA-3072. Prevents firmware replacement attacks.
Hardware Root of Trust	TPM (Trusted Platform Module) or HSM (Hardware Security Module) chip generates and stores cryptographic keys in tamper-resistant hardware. Keys never exposed in software memory. Used in enterprise IoT devices.
Flash Encryption	Encrypt the device's flash memory. Even with physical access (JTAG, SPI flash reading), firmware and keys are unreadable. ESP32 supports AES-128/256 flash encryption.
Unique Device Identity	Each device gets a unique identity at manufacturing (certificate, private key burned into OTP fuses). Prevents cloning and impersonation. AWS IoT Greengrass supports fleet provisioning with unique certificates.
Secure Element (SE)	Dedicated security chip (Microchip ATECC608, NXP SE050) handling all cryptographic operations — key generation, ECDH, ECDSA, SHA. Hardware-isolated from main MCU.
Physical Tamper Protection	Tamper detection (accelerometer detecting physical movement, enclosure contact sensors) that wipes keys and logs tamper events. Epoxy-filled enclosures prevent circuit board access.

Watchdog Timer	Hardware timer that resets device if firmware hangs (e.g., due to buffer overflow exploit). Ensures device availability even under attack.
-----------------------	--

6.3.3 Communication Security

Protocol	Security Mechanism
MQTT over TLS (MQTTS)	TLS 1.3 on port 8883. X.509 certificate-based mutual authentication (mTLS) — both client and server verify each other's certificates. Prevents MitM. All MQTT payload encrypted.
HTTPS	TLS 1.3 on port 443. Server certificate verified by client. For IoT REST APIs, also add API key (X-API-Key header) or Bearer token (JWT) for authorisation.
CoAP + DTLS	DTLS (Datagram TLS) for CoAP over UDP. Provides same security properties as TLS but for connectionless transport.
Wi-Fi Security	WPA3-Personal: SAE (Simultaneous Authentication of Equals) replaces PSK — resistant to offline dictionary attacks. WPA3-Enterprise: 802.1X with RADIUS server for certificate-based authentication.
Zigbee Security	AES-128 encryption at both network and application layers. Network key protects in-network communication. Application link key protects end-to-end device-to-device communication.
LoRaWAN Security	AES-128 MIC (Message Integrity Code) provides authenticity. NwkSKey encrypts network data. AppSKey encrypts application payload. Both OTAA and ABP, but OTAA is more secure.
BLE Security	LE Secure Connections (BLE 4.2+): ECDH key exchange for pairing. AES-128 CCM for payload encryption. Passkey/OOB pairing prevents impersonation.

6.3.4 IoT Privacy Principles — GDPR and IoT

GDPR/Privacy Principle	IoT Application
Data Minimisation	Collect ONLY the data necessary for the stated purpose. Smart home device should not collect voice recordings 24/7 if wake word detection can be done on-device.
Purpose Limitation	Data collected for one purpose (health monitoring) cannot be used for another (insurance profiling) without new consent.
Storage Limitation	Delete IoT data when no longer needed. Implement automatic data retention policies (InfluxDB retention policies, S3 lifecycle rules deleting after 90 days).
Transparency	IoT devices must clearly disclose what data they collect, how it is used, and who it is shared with. Smart speaker should have physical light indicator when microphone is active.

User Rights	Right to access, rectify, and erase personal data. IoT systems must provide mechanisms to retrieve a user's device data or delete their account and all associated data.
Data Localisation	Some regulations (Russia, China, healthcare) require personal data to remain within national borders. Cloud provider region selection + private cloud deployment.
Privacy by Design	Security and privacy considered from system inception, not added after. Encrypt before sending; anonymise before analytics; use differential privacy for aggregate statistics.
Breach Notification	GDPR requires notifying supervisory authority within 72 hours of discovering a data breach affecting personal data. IoT security monitoring and SIEM required.

7. Quick Revision Summary & Important Exam Questions

7.1 Quick Revision Table — All Unit IV Concepts

Topic	Key Points to Remember
Cloud Computing	On-demand IT resources over Internet. IaaS (VMs), PaaS (managed platform), SaaS (full app), FaaS (serverless functions). Public/Private/Hybrid/Multi-cloud.
ThingSpeak	MathWorks. Channel = up to 8 fields. Write/Read API Keys. Free: 15s update limit. MATLAB analysis, React rules, TalkBack queue for commands. HTTP GET and MQTT supported.
AWS IoT Core	MQTT broker + Device Registry + Device Shadow + Rules Engine. X.509 certs. Port 8883 (TLS). Rules SQL query → DynamoDB/Lambda/S3/SNS/Kinesis. Greengrass for edge.
Device Shadow	JSON document with desired (app wants) and reported (device is) states + delta (difference). Persists when device offline. MQTT topics: \$aws/things/{name}/shadow/...
Google Cloud IoT	IoT Core deprecated 2023. Data pipeline: Device → Pub/Sub → Dataflow → BigQuery. Firebase for real-time app dashboards. Vertex AI for ML on IoT data.
5 Vs of IoT Data	Volume (massive), Velocity (real-time streams), Variety (structured/unstructured), Veracity (quality/accuracy), Value (business insights extracted).
Stream Processing	Real-time analysis as data arrives. Windowing (tumbling/sliding/session). Flink, Kafka Streams, Kinesis Analytics, Azure Stream Analytics.
Batch vs Stream	Batch: stored historical data, minutes-hours latency, Spark/Hadoop. Stream: in-motion data, ms-sec latency, Flink/Kinesis. IoT needs both (Lambda Architecture).
Time-Series DB	Purpose-built for timestamped sensor data. InfluxDB: measurement/tag/field/retention policy. Tags indexed (fast filter). Fields not indexed (actual values).
InfluxDB Concepts	Measurement=table, Tag=indexed metadata (device_id, location), Field=actual value (temp=24.5), Retention Policy=auto-delete after N days.
Grafana	Open-source dashboard. Connects to InfluxDB, Prometheus, MySQL, etc. Panels: time series, stat, gauge, heatmap. Alerting to Slack/email/webhook.
Edge Computing	Processing near data source. <5ms latency. Reduces bandwidth. Offline operation. TinyML/TF Lite Micro on MCU. 3 levels: device, gateway, micro DC.
Fog Computing	Cisco 2014. Intermediate layer between device and cloud. Fog nodes = smart gateways. More capability than edge device, less centralised than cloud.
Edge vs Fog vs Cloud	Edge: on device (<5ms, no internet needed). Fog: intermediate gateway (1-50ms, local analytics). Cloud: remote DC (50-500ms, unlimited scale, requires internet).

IoT Security Challenges	Resource constraints, scale (billions of devices), long lifetime, physical access, default credentials, lack of updates.
Authentication	X.509 certs (AWS IoT), JWT tokens (Google Cloud IoT), SAS tokens (Azure IoT). Hardware: TPM, HSM, Secure Element (ATECC608).
Encryption	TLS 1.3 for MQTT (port 8883) and HTTPS. DTLS for CoAP. AES-128 for Zigbee. AES-128 NwkSKey+AppSKey for LoRaWAN. WPA3 for Wi-Fi.
Common Attacks	Mirai botnet (default creds), MitM (no TLS), replay attack (no nonce), firmware attack (no secure boot), eavesdropping (unencrypted MQTT).
GDPR in IoT	Data minimisation, purpose limitation, storage limitation, transparency, user rights (access/delete), privacy by design, 72hr breach notification.
Secure Boot	Cryptographic firmware signature verification before execution. ESP32 supports it. Prevents firmware replacement attacks.

7.2 Important Exam Questions with Detailed Answer Outlines

2-Mark Questions

Q1. What is ThingSpeak? List its key features.

Answer: ThingSpeak = MathWorks IoT analytics platform. Features: Channels (up to 8 fields), Write/Read API Keys, HTTP and MQTT support, MATLAB analysis, React (condition-action rules), TalkBack (command queue), free tier with 15s update limit.

Q2. What is a Device Shadow in AWS IoT Core?

Answer: A Device Shadow is a persistent JSON document in AWS IoT Core representing a device's desired (what app wants) and reported (what device actually is) state. Persists when device is offline. Delta field shows mismatch between desired and reported. Enables cloud-to-device communication even when device is offline.

Q3. Differentiate between Edge Computing and Fog Computing.

Answer: Edge: Processing on device or 1-hop gateway. Latency <5ms. Can work offline. TinyML on MCU. Fog: Intermediate processing nodes between devices and cloud (Cisco 2014). Latency 1-50ms. More capable than edge nodes. Acts as regional processing layer. Both reduce cloud dependency.

Q4. What is InfluxDB? What are its key concepts?

Answer: InfluxDB = open-source purpose-built time-series database. Key concepts: Measurement (like table), Tag (indexed metadata — device_id, location), Field (actual values — temperature, humidity), Timestamp (nanosecond precision), Retention Policy (auto-delete after N days).

Q5. List any four major IoT security threats.

Answer: 1) Botnet/DDoS (Mirai — default credentials), 2) Man-in-the-Middle (no TLS encryption), 3) Replay Attack (re-sending captured valid commands), 4) Firmware Attack (replacing firmware via OTA hijacking or supply chain). Also: eavesdropping, physical tampering.

Q6. What is secure boot in IoT devices?

Answer: Secure Boot is a security mechanism that cryptographically verifies the digital signature of firmware before executing it. If firmware is tampered or replaced by an attacker, the signature verification fails and the device refuses to boot. ESP32 supports Secure Boot V2 using RSA-3072. Prevents firmware replacement attacks.

4-Mark / 5-Mark / 6-Mark Questions — Detailed Answer Outlines

Q1. Explain AWS IoT Core in detail — components, Device Shadow, Rules Engine, and security.

Answer outline — cover these points:

1. Components: Message Broker (MQTT/HTTPS/WebSocket), Device Registry (thing catalog), Device Shadow (desired/reported/delta JSON), Rules Engine (SQL-based, 10+ actions), IoT Policies (permissions), Fleet Indexing.
2. Device Shadow: JSON with desired (app wants), reported (device is), delta (difference). Persists offline. MQTT topics: \$aws/things/{name}/shadow/update|get|delete|update/delta|update/accepted|rejected.
3. Rules Engine: SQL query on MQTT stream. SELECT temp FROM sensors/+ WHERE temp > 30 → Lambda + DynamoDB + SNS simultaneously. 10+ action types.
4. Security: X.509 certificates for each device (mTLS). IoT Policies for MQTT topic permissions. TLS 1.3 on port 8883. VPC endpoints, CloudTrail logging, Device Defender.
5. Python connection example: `mtls_from_path()` with certificate, private key, and CA root certificate. Publish JSON to topics, subscribe with callbacks.

Q2. Explain ThingSpeak with its architecture, sending/receiving data methods, and MATLAB analytics.

Answer outline — cover these points:

1. Architecture: Channels (up to 8 fields), Write API Key (secret, device writes), Read API Key (read-only, share publicly), Channel Feed (time-series data log).
2. Sending data: HTTP GET → `api.thingspeak.com/update?api_key=KEY&field1=VALUE&field2=VALUE`. MQTT: broker `mqt3.thingspeak.com`, topic `channels/CHANNEL_ID/publish`, payload `field1=val&field2=val`.
3. Reading data: GET `/channels/ID/feeds.json?results=N`. Returns JSON with `entry_id`, `created_at`, `field1..field8`. Date range, field-specific, and last-entry endpoints available.
4. MATLAB Analysis: Execute MATLAB code triggered by new data, timer, or on-demand. Statistical analysis, visualisation, signal processing. Results saved to a channel.
5. React rules: IF `field1 > 30` THEN HTTP request (IFTTT, webhook). TalkBack: command queue polled by device. TimeControl: scheduled code execution.

6. Free tier: 4 channels, 15s update minimum, 3M stored messages, 60 min/day MATLAB.

Q3. Compare Edge Computing, Fog Computing, and Cloud Computing with examples and use cases.

Answer outline — cover these points:

1. Cloud: Centralised DC. 50-500ms latency. Unlimited scale. Requires internet. Use: Historical analytics, ML training, global dashboards. Example: AWS IoT Core, Google BigQuery.
2. Fog: Intermediate nodes (Cisco 2014). 1-50ms latency. Regional processing. Local analytics + caching. Use: Smart city infrastructure, industrial IoT control. Example: 5G MEC, smart gateway with local MQTT broker.
3. Edge: On device or 1-hop gateway. <5ms latency. Offline capable. TinyML, local rules. Use: Autonomous vehicles, surgical robots, CNC machines. Example: ESP32 TF Lite, NVIDIA Jetson Nano.
4. Comparison table: location, latency, bandwidth, scale, offline, privacy, examples.
5. Data flow: Raw data → Edge (immediate action) → Fog (aggregate/filter) → Cloud (long-term analytics). Lambda/Kappa Architecture principles.
6. TinyML: TF Lite Micro on MCU. Keyword detection, anomaly detection without cloud round-trip.

Q4. Explain IoT security challenges, major attack types, and security best practices.

Answer outline — cover these points:

1. Challenges: Resource constraints (no full TLS on MCU), scale (billions of devices), long lifetime (15-20 years), physical access, default credentials, lack of patches.
2. Attacks: Mirai botnet (default creds DDoS), MitM (no TLS), replay attack (no nonce), firmware attack (no secure boot), eavesdropping (unencrypted), physical tampering.
3. Device security: Secure Boot (firmware signature), Flash Encryption, Secure Element (ATECC608), TPM, Watchdog Timer, unique identity per device.
4. Communication security: TLS 1.3 (MQTT port 8883, HTTPS), DTLS (CoAP), WPA3, Zigbee AES-128, LoRaWAN NwkSKey+AppSKey.
5. 7 Pillars: Authentication, Authorisation (least privilege), Encryption in transit, Encryption at rest, Integrity (signatures), Availability, Privacy.
6. GDPR: Data minimisation, purpose limitation, storage limitation, transparency, user rights, 72hr breach notification, privacy by design.

Q5. Explain data storage options, InfluxDB, and data visualisation tools for IoT.

Answer outline — cover these points:

1. Storage types: Time-series DB (InfluxDB, Timesstream — best for IoT), NoSQL Document (MongoDB/DynamoDB — flexible schema), Wide-Column (Cassandra/Bigtable — massive scale), In-memory (Redis — current value cache), Data Lake (S3/GCS — raw archival).

2. InfluxDB: Measurement (table), Tag (indexed metadata), Field (actual values), Timestamp (nanosecond), Retention Policy (auto-delete). Line protocol for writing. Flux/InfluxQL for querying.
3. InfluxDB queries: `from(bucket:'iot') |> range(start:-24h) |> filter(fn:(r) => r._measurement == 'sensors') |> mean()`. Continuous queries for downsampling.
4. Grafana: Open-source dashboard. Data sources (InfluxDB, Prometheus, MySQL). Panel types (time series, stat, gauge, heatmap, geomap). Alerting to Slack/email. Variables for dynamic filtering.
5. Other tools: ThingSpeak (beginner), Node-RED (visual flow), Blynk (mobile), Kibana (ELK), Power BI (enterprise), Looker Studio (Google BigQuery).

⚡ Key Points to Remember

- ThingSpeak: Channels (8 fields) + Write/Read API Keys. Free = 15s update limit. MATLAB + React + TalkBack.
- AWS IoT Core: Device Shadow = desired/reported/delta JSON. Rules Engine = SQL query on MQTT → DynamoDB/Lambda/SNS. Port 8883 TLS.
- Device Shadow MQTT: `$aws/things/{name}/shadow/update | get | delete | update/delta`
- InfluxDB: Measurement=table, Tag=indexed (device_id), Field=value (temp=24.5), Retention Policy=auto-delete.
- Edge (<5ms, on-device, offline) vs Fog (1-50ms, intermediate, Cisco 2014) vs Cloud (50-500ms, unlimited scale).
- TinyML: TF Lite Micro for on-device ML. Edge Impulse for training + deployment. Coral Edge TPU hardware accelerator.
- IoT Security: Mirai botnet (default creds), Secure Boot (firmware signature), TLS port 8883, AES-128 Zigbee/LoRa.
- GDPR: Data minimisation, purpose limitation, storage limitation, transparency, 72hr breach notification, privacy by design.